

Inferencing

Friday, 26 September 2025 6:16 PM

- Transformer Architecture
 - Tokenization
 - Embedding
 - Static
 - Positional Encoding
 - Layer Normalization
 - Self-attention
 - Feed forward Network
 - Activation
 - Logits
 - Softmax
 - Decoding
 - Auto-Regressive
-
- Capacity Planning
 - Desired Inputs
 - Input tokens / sec
 - Output tokens / sec
 - Requests / sec
 - Concurrent users
 - Model Families
 - TTFT
 - TPOT
 - Additional Info
 - Tenant Priority and Category
 - Geographical distribution
 - Available hardware (GPU , Server etc)
 - Separate decode and Prefill calculation
-
- Deployment Topology
 - Control Plane
 - User/Tenant Registration
 - Model Registration
 - Quotas
 - Token Budget
 - Request Rate
 - Policies
 - Routing Table
 - Gateway (Proxy)
 - Authentication & Authorization
 - Routing
 - Rate Limiting
 - Graceful Degradation
 - Safety Filters
 - Async Plane (Optional)
 - Data Plane
 - Accelerator Plane
 - Host
 - CPU Isolation
 - NUMA placement
 - IRQ affinity
 - Memory
 - Huge Page Configuration
 - Networking
 - RDMA
 - NCCL
 - Ethernet
 - Storage
 - NVME
 - Kubernetes
 - Node Pools
 - Taint and Toleration
 - Affinity and AntiAffinity
 - Resource and Limit
 - Scaling Strategies

- Scaling Strategies
 - MIG Partitioning
 - Kublet configuration
 - cpuManagerPolicy
 - Huge Pages mounting with PVC
 - Device Plugins
- Tooling
 - Model Registry
 - LLM Runtime image repo
 - Key Management
 - Deployment config (ArgoCD + Git)

- Optimization
 - System
 - ◆ CPU Isolation
 - GRUB_CMDLINE_LINUX="isolcpus=4-19,24-39 nohz_full=4-19,24-39 rcu_nocbs=4-19,24-39"
 - NUMA Pinning
 - ◆ Host :- numactl --cpunodebind=0 --membind=0 -- ./fraud-scorer.
 - ◆ K8 :- CPU Manager, kind: ClusterConfiguration and kind: KubeletConfiguration with static policy
 - ◆ K8 :- Resource and Limits
 - Memory
 - HugePages on Host
 - Shared Memory (/dev/shm)
 - Volume with HugePage and shared memory (IPC Namespace) on K8
 - Network
 - RDMA
 - SR-IV CNI plugging. RDMA access via SR-IOV.
 - Ethernet
 - Cillium
 - OS Kernel overhead
 - AF_XDP (Instead DPDK due to high operation challenges)
 - NIC Ring Buffer and Multi-Queue configuration . iouring
 - GPU
 - Memory Pinning
 - Storage
 - PVC with NVME Local Model
 - LLM Runtime
 - Optimization
 - Standard
 - Batching
 - ◆ Static (Request Base or Time Based)
 - ◆ Continuous (Micro Batches)
 - Paged Attention (Inference Engine like vLLM) / Radix Attention (SgLang)
 - Quantization
 - ◆ K-V quantization
 - ◆ Weight Quantization
 - ◆ Activation Quantization
 - Prefix Caching
 - Session Affinity
 - Time Slicing or CUIDA MPS (Multi Process Service) or MIG
 - Advanced
 - Speculative Decoding
 - ◆ Token speculation
 - ◆ Parallel verification
 - ◆ Rejection sampling
 - Separate Prefill-Decode phase
 - ◆ llm-d
 - ◆ NVIDIA Dynamo
 - Pruning (Training Phase)
 - ◆ Structured Pruning (Remove Layer)
 - ◆ Unstructured Pruning(Remove Neurons)
 - ◆ Activation based
 - Custom Kernel (Fusion to reduce memory bandwidth)
 - Flashed Attention , Grouped Query and Multi-Query Attention(Implement in Model Code)

- Observability
 - Areas
 - Gateway
 - Request rate, Errors, p99/p95/p50, Queueing, Concurrent sessions
 - LLM Runtime
 - TTFT, TPOT, batching, KV cache, prefill/decode, token rate
 - Cluster (Kubernetes)
 - Node health, Autoscaling, Pod Health, Readiness and Liveliness Probe
 - Host
 - CPU, Memory, Network (Ethernet, RDMA & NCCL)
 - GPU
 - Utilization, Occupancy, HBM

- Utilization, Occupancy , PBM
- Functional
 - Cost , Safety, Hallucination, instruction adherence
- Telemetry
 - eBPF
 - OTEL
- Tools
 - CPU
 - Top/htp, uptime, ps
 - Memory
 - Free, vstat
 - Network
 - ss, netstat
 - NUMA
 - numastat
 - Storage
 - Iostat, iotop
 - GPU
 - Nvidia-smi
- Benchmarking
 - CPU memory bandwidth
 - STREAM
 - CPU/GPU compute sanity
 - HPL
 - GPU
 - DGCM Diag
 - Storage
 - FIO
 - Network
 - NCCL tests
 - End_To_End
 - MLPerf
- Profiling
 - CPU
 - Perf
 - Intel vTune
 - Memory
 - Heaptrack
 - tcmalloc
 - Network
 - Wireshark
 - GPU
 - NVIDIA Nsight
 - Nvidia Compute

- Operation Excellence
 - Multi-tenant: per-tenant adapters, JWT/RBAC, quotas/rate limits, per-tenant KV isolation, cost tagging, noisy neighbor controls.
 - Autoscaling: GPU node pools by model size; bin-packing by VRAM; cold-start strategies; queueing + SLO-aware admission.
 - Resilience: health checks by token progress, backpressure, circuit breakers, canary deploys; blue/green for drivers/kernels.
 - Observability: per-request tokens, TTFT/TBT, queue times, batch size histograms, cache hit rates, OOMs, evictions, cost per 1k tokens.
 - Define SLOs (p90 TTFT, p95 latency, availability) and enforce via admission control.
 - Keep models hot (warm pools) for popular SKUs; load adapters lazily.
 - Separate gateway (auth, rate limit, routing) from GPU workers.
 - Always instrument and budget latency—optimize the biggest slice first.
 - Keep compat contracts: prompt/response schema, safety headers, model version pins.
 - pinned memory, larger batch windows (with max wait), speculative/medusa heads, flash-/paged-attention, CUDA streams overlap.
- Governance
 - Cost
 - Platform Readiness
 - Model Readiness
- Security
- Operation Challenges
 - Reliability
 - High availability
 - Scaling
 - Multi Cloud/Multi-Cluster inferencing
 - Security
 - Privacy Preserving
 - Cost

- LLM Runtimes
 - vLLM

- max_model_len
- max_num_batched_tokens
- max_num_seqs
- enable_chunked_prefill
- enable_prefix_caching
- tensor_parallel_size
- gpu_memory_utilization

- SgLang
- Titon + TensorRT-LLM

- Customization

- Interview Questions

- **Transformer and autoregressive basics**

- 1. Walk me through what happens when a user sends a prompt to a decoder-only LLM.**

Expected points: tokenization, embeddings, transformer layers, self-attention, FFN, logits, sampling, next-token generation, autoregressive loop.

- 2. What is the difference between prefill and decode in LLM inference?**

Expected points: prefill processes all input tokens in parallel; decode generates one token per sequence step; prefill is compute-heavy, decode is memory/KV-cache-heavy.

- 3. Why is LLM generation called autoregressive?**

Expected points: each new token depends on previous tokens; model appends generated token back into context; cannot generate all output tokens fully in parallel.

- 4. What are logits, and how do temperature, top-k, and top-p affect decoding?**

Expected points: logits are raw scores before softmax; temperature controls randomness; top-k limits candidates by count; top-p limits candidates by cumulative probability.

- 5. Why does attention become expensive for long-context prompts?**

Expected points: attention compares tokens with previous tokens; prefill cost grows with sequence length; KV cache memory grows with context length and active sequences.

- **Latency, throughput, and serving lifecycle**

- 6. Explain the full latency budget of an LLM request.**

- 7. What is TTFT, and why does it matter?**

Expected points: Time To First Token; user-perceived responsiveness; affected by queueing, prompt length, prefill, batching, scheduling, cold starts.

- 8. What is TPOT or inter-token latency?**

Expected points: Time Per Output Token; decode performance; affected by KV cache bandwidth, batch size, sampling overhead, memory bandwidth.

- 9. Why can P50 latency look good while P99 latency is bad?**

Expected points: queueing, long prompts, KV pressure, GPU memory fragmentation, noisy neighbors, batching delays, cold starts, outlier requests.

- 10. How would you debug a sudden increase in P99 TTFT after prompt length increases?**

Expected points: isolate queue wait vs prefill; profile input length buckets; check GPU memory, KV cache allocation, batch composition, prefill kernels, attention implementation.

- **Batching and scheduling**

- 11. What is the difference between static batching, dynamic batching, and continuous batching?**

Expected points:

- 12. Why is continuous batching important for LLM serving?**

Expected points: requests finish at different times; decode length varies; continuous batching keeps GPU busy by admitting new requests without waiting for entire batch to finish.

- 13. How does chunked prefill work, and why is it useful?**

Expected points: split long prompt prefill into chunks; interleave prefill and decode; reduce head-of-line blocking; improve responsiveness for mixed workloads.

- 14. What is request preemption in LLM serving?**

Expected points: pause/evict lower-priority sequence; free KV cache or scheduling slot; useful for priority traffic, overload, long requests.

- 15. How do you design backpressure for an LLM gateway?**

Expected points: bounded queues, admission control, max prompt tokens, max output tokens, tenant quotas, queue timeout, 429/503, degrade modes.

- **KV cache, memory, and attention optimization**

- 16. What is KV cache, and why is it critical for decode performance?**

Expected points: stores key/value tensors from previous tokens; avoids recomputing past attention; consumes large GPU memory; decode reads it every token.

- 17. What problem does PagedAttention solve?**

Expected points: KV cache fragmentation; variable sequence lengths; page/block-based KV allocation; better memory utilization; enables high concurrency.

- 18. Compare PagedAttention and RadixAttention / prefix caching.**

Expected points: PagedAttention manages KV memory efficiently; RadixAttention/prefix caching reuses shared prefixes; useful for repeated system prompts, agents, RAG templates.

- 19. What is FlashAttention, and where does it help?**

Expected points: optimized attention kernel; reduces HBM traffic; uses tiling and recomputation; more useful during prefill/attention-heavy phases.

- 20. What is the difference between MHA, MQA, and GQA?**

- **Model/runtime optimization**

- 21. How does speculative decoding improve throughput?**

Expected points: small draft model proposes tokens; large model verifies in parallel; accepted tokens reduce expensive

decode steps; rejection sampling maintains output distribution.

22. How does quantization affect LLM inference?

Expected points: lower memory footprint, higher throughput, possible accuracy loss; INT8/INT4; weight-only vs activation quantization; kernel support matters.

23. When would you use TensorRT-LLM over vLLM?

Expected points: TensorRT-LLM for highly optimized NVIDIA deployment, quantization, fused kernels, static/optimized engines; vLLM for flexible serving, PagedAttention, continuous batching, easier ops.

24. Where does Triton Inference Server fit compared with vLLM, TensorRT-LLM, and ONNX Runtime?

Expected points: Triton is serving layer; can host multiple backends; useful for model repository, batching, metrics, versioning; vLLM/TensorRT-LLM/ONNX Runtime are execution/runtime backends or engines.

25. Design a production LLM inference platform for multi-tenant traffic. What are the main components?

- Safety / Guardrail